Experiment 02B

Medical GPT Training with a Custom Biomedical BPE Tokenizer

Medical text corpus | Custom biomedical BPE | Standard Attention | Lightning H200 | 5 epochs

Report summary

This report documents the custom biomedical BPE tokenizer experiment following the scaled GPT-2/tiktoken baseline. Experiment 02B replaces the generic tokenizer with a domain-specific tokenizer while preserving a comparable medical-domain training setup, producing a substantially lower validation loss and perplexity.

Tokenizer	Custom biomedical BPE
Token Count	~5M tokens
Model Scale	~194M parameters
Context Length	1024 tokens
Final Val Loss	2.1301
Final Perplexity	8.4161
Status	Tokenizer comparison baseline

1. Objective

Experiment 02B evaluates whether a custom biomedical BPE tokenizer improves medical-domain language modeling compared with the GPT-2/tiktoken baseline. The model remains a GPT-style decoder-only Transformer trained on the same domain setting with standard causal self-attention.

The experiment isolates tokenizer choice as the primary design change. The key question is whether biomedical-domain subword segmentation reduces validation loss and perplexity relative to a generic tokenizer.

2. Experiment Configuration

Component	Configuration
Dataset	Medical text corpus
Tokenizer	Custom biomedical BPE
Vocabulary size	52,000
Architecture	GPT-style decoder-only Transformer
Attention	Standard causal multi-head self-attention
Context length	1024 tokens
Model scale	~194M parameters
Hardware	Lightning AI H200
Epochs	5

3. Training Configuration

Hyperparameter	Value
Optimizer	AdamW
Learning rate	3e-4
Minimum learning rate	1e-5
Scheduler	Warmup + cosine decay
Batch size	8
Gradient accumulation	8 steps
Effective batch size	64
Mixed precision	Enabled
Checkpointing	best.pt, latest.pt, epoch_N.pt

4. Training Results

Epoch	Optimizer Step	Train Loss	Validation Loss	Perplexity	Learning Rate
1	420	4.122820	3.588786	36.1901	2.84e-04
2	840	3.043306	2.642536	14.0488	2.15e-04
3	1260	2.464203	2.313835	10.1131	1.20e-04
4	1680	2.204899	2.170586	8.7634	4.06e-05
5	2100	2.087899	2.130147	8.4161	1.00e-05

Validation loss decreased from 3.5888 to 2.1301, while perplexity decreased from 36.1901 to 8.4161. This represents stable convergence and a strong improvement over the generic-tokenizer baseline.

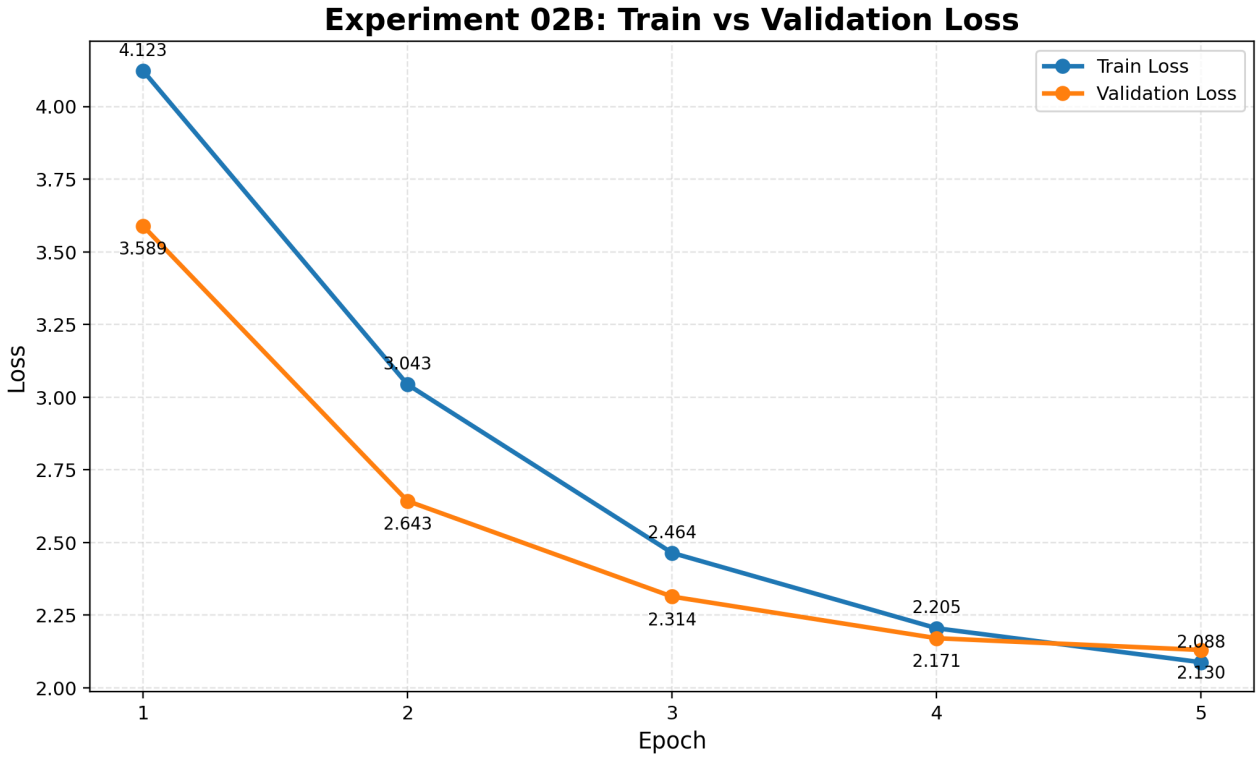

Figure 1. Training and validation loss across epochs.

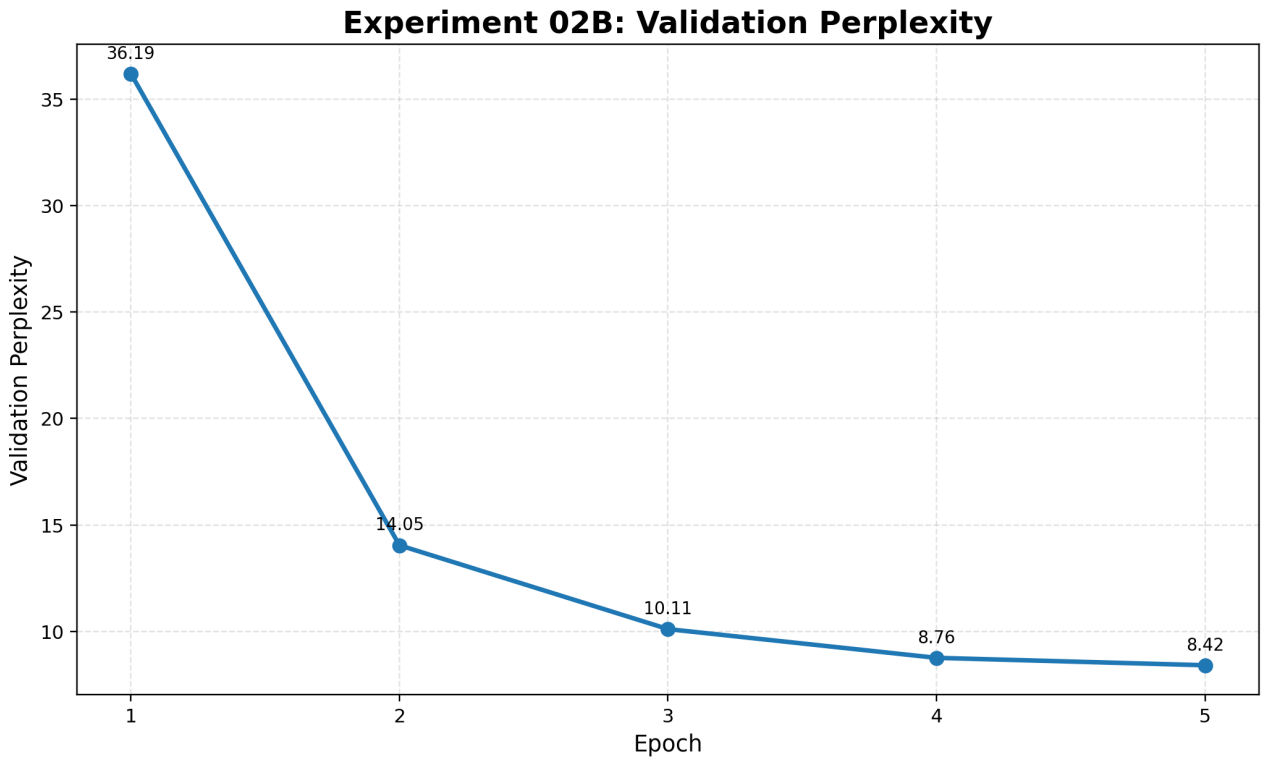

Figure 2. Validation perplexity across epochs.

5. Learning Rate and Generalization Behavior

The learning rate followed a decay schedule after warmup, while validation performance improved across training. The generalization gap remained controlled and did not show the severe overfitting pattern observed in the initial small-scale baseline.

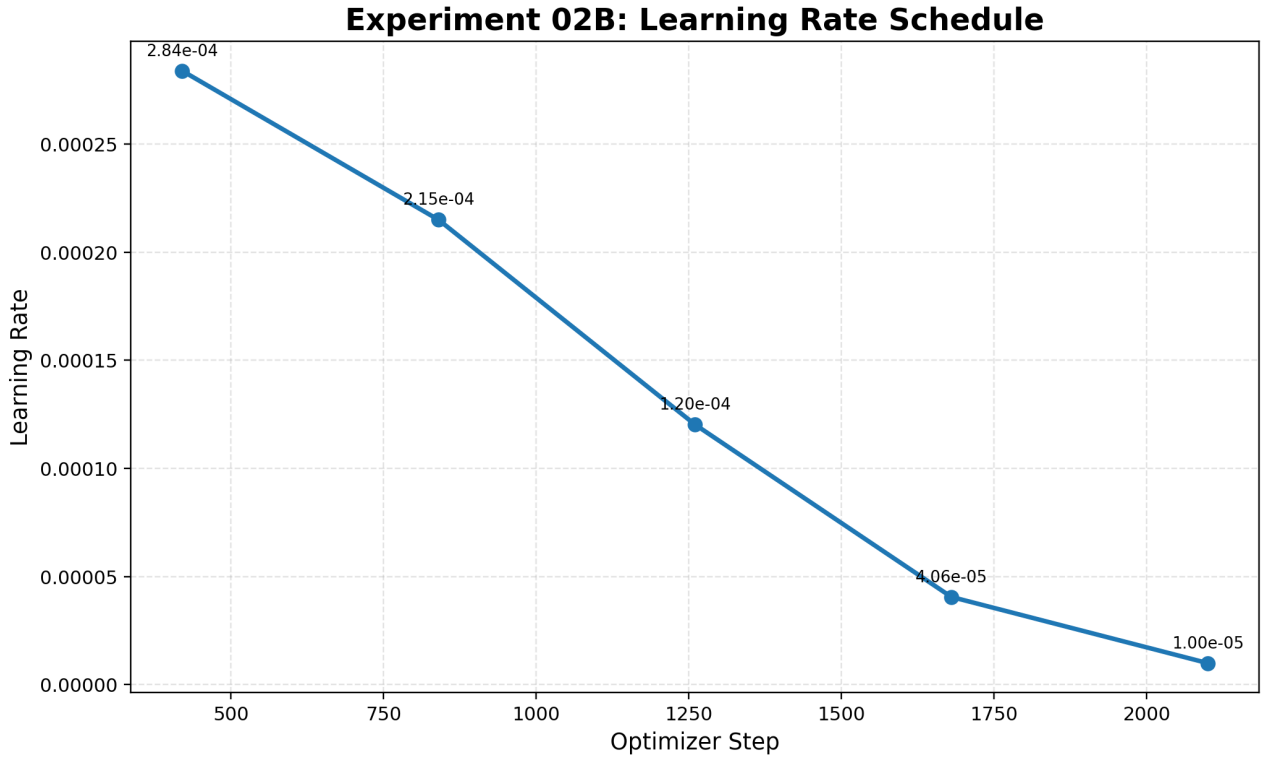

Figure 3. Learning rate schedule across optimizer steps.

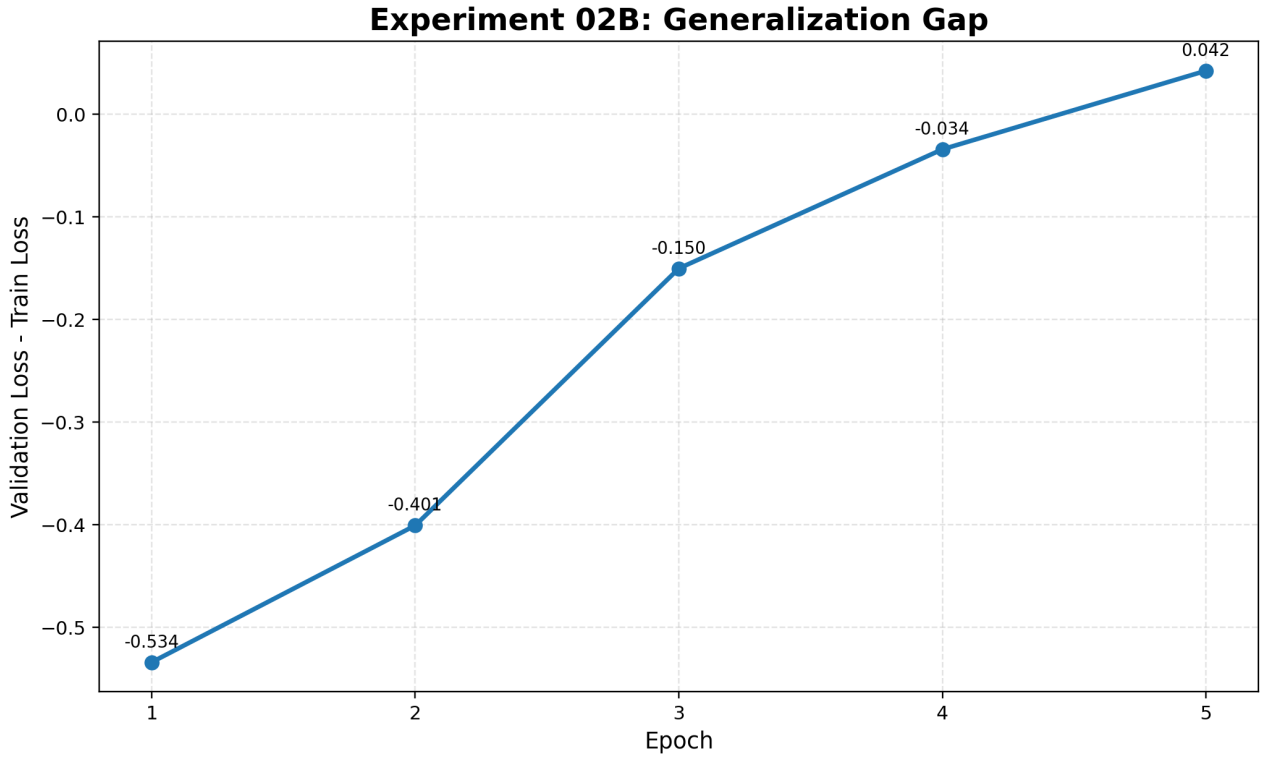

Figure 4. Generalization gap measured as validation loss minus training loss.

6. KV Cache Benchmark

Inference benchmarking was conducted with and without key-value caching for prompt lengths of 64, 128, 256, and 512 using 64 generated tokens in each case. To preserve readability, the benchmark is reported as two compact tables: latency/throughput and memory usage.

Prompt Length	Max New Tokens	Latency w/o KV	Latency w/ KV	Speedup	TPS w/o KV	TPS w/ KV
64	64	0.339639	0.308528	1.1008x	188.45	207.45
128	64	0.351796	0.310532	1.1329x	182.92	206.13
256	64	0.338507	0.311260	1.0875x	189.07	205.62
512	64	0.452341	0.303662	1.4896x	141.49	210.76

Table 1. KV-cache latency and throughput benchmark.

Prompt Length	Peak Memory w/o KV (MB)	Peak Memory w/ KV (MB)	Memory Difference (MB)
64	823.13	795.98	27.15
128	848.32	829.60	18.72
256	900.70	867.82	32.88
512	1001.83	975.28	26.55

Table 2. KV-cache peak memory usage.

The strongest KV-cache speedup was observed at prompt length 512, where latency improved from 0.452341 seconds to 0.303662 seconds, corresponding to a 1.4896x speedup.

7. KV Cache Plots

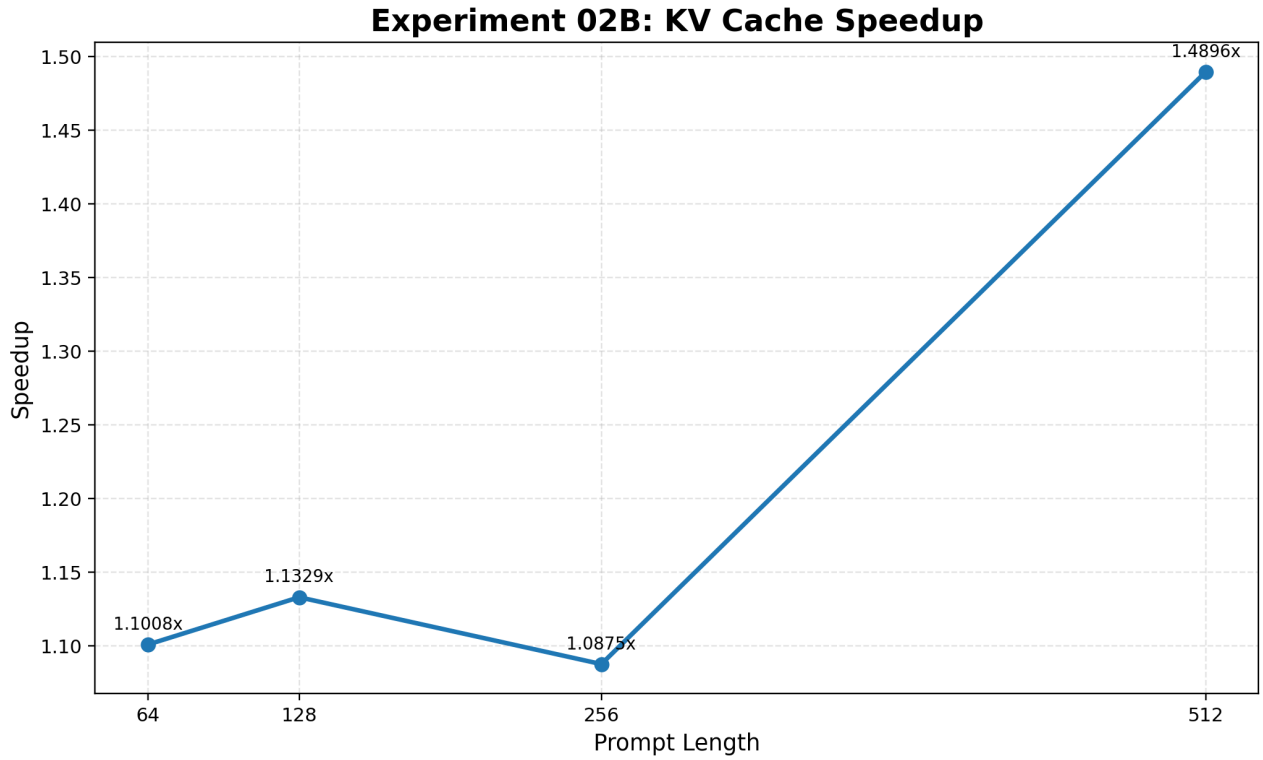

Figure 5. KV-cache speedup across prompt lengths.

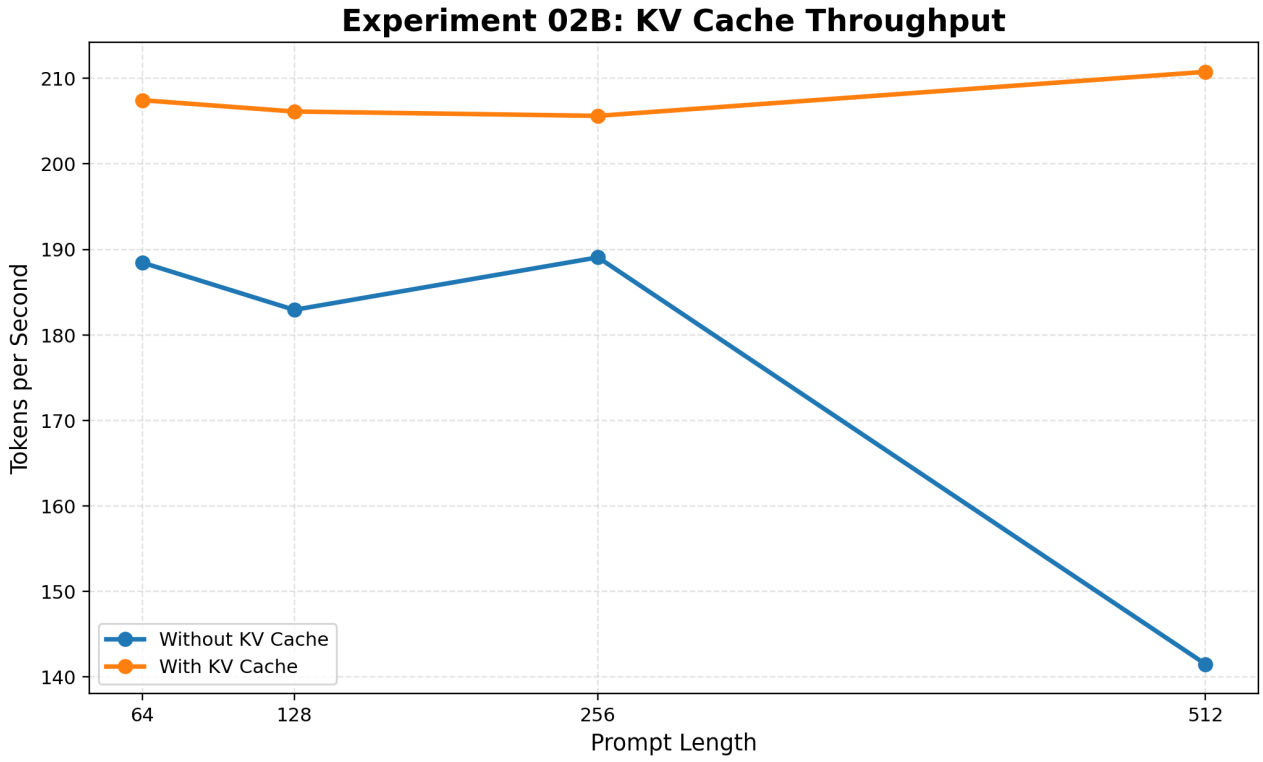

Figure 6. Throughput comparison with and without KV-cache.

8. Additional Inference Metrics

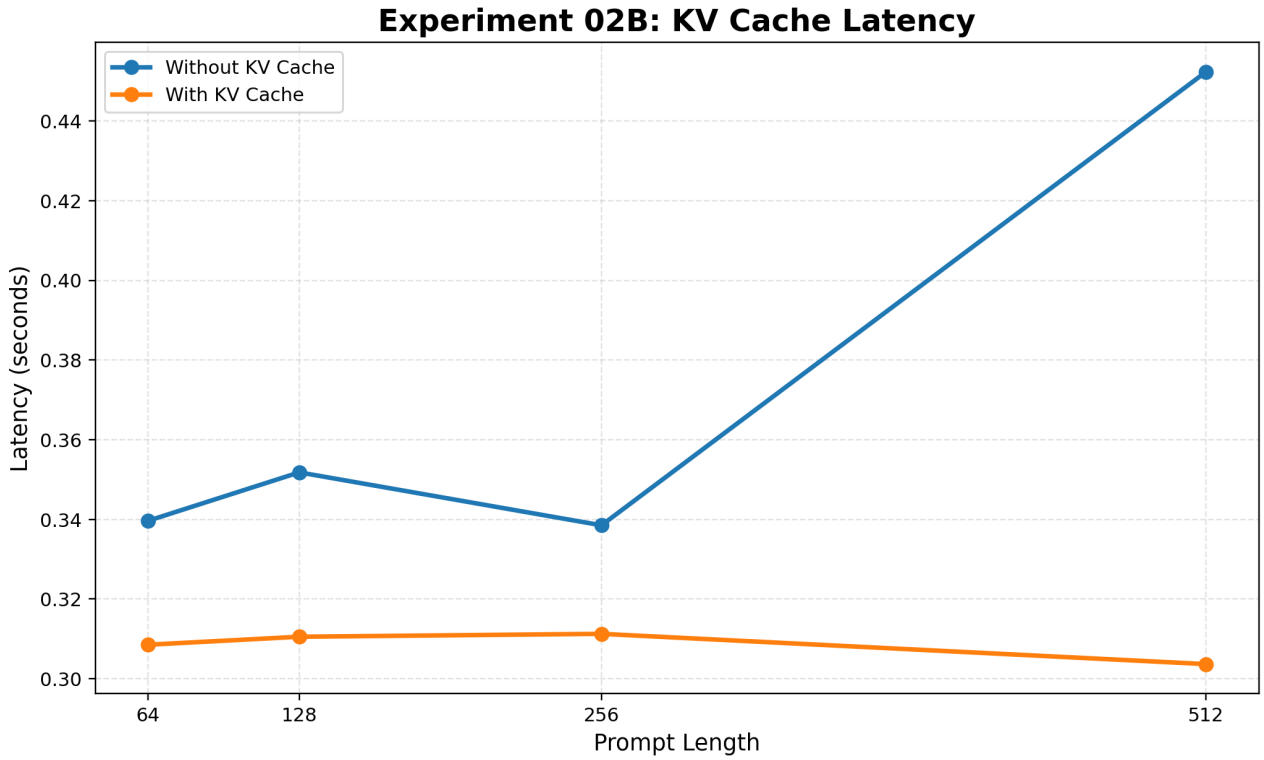

Figure 7. Latency comparison with and without KV-cache.

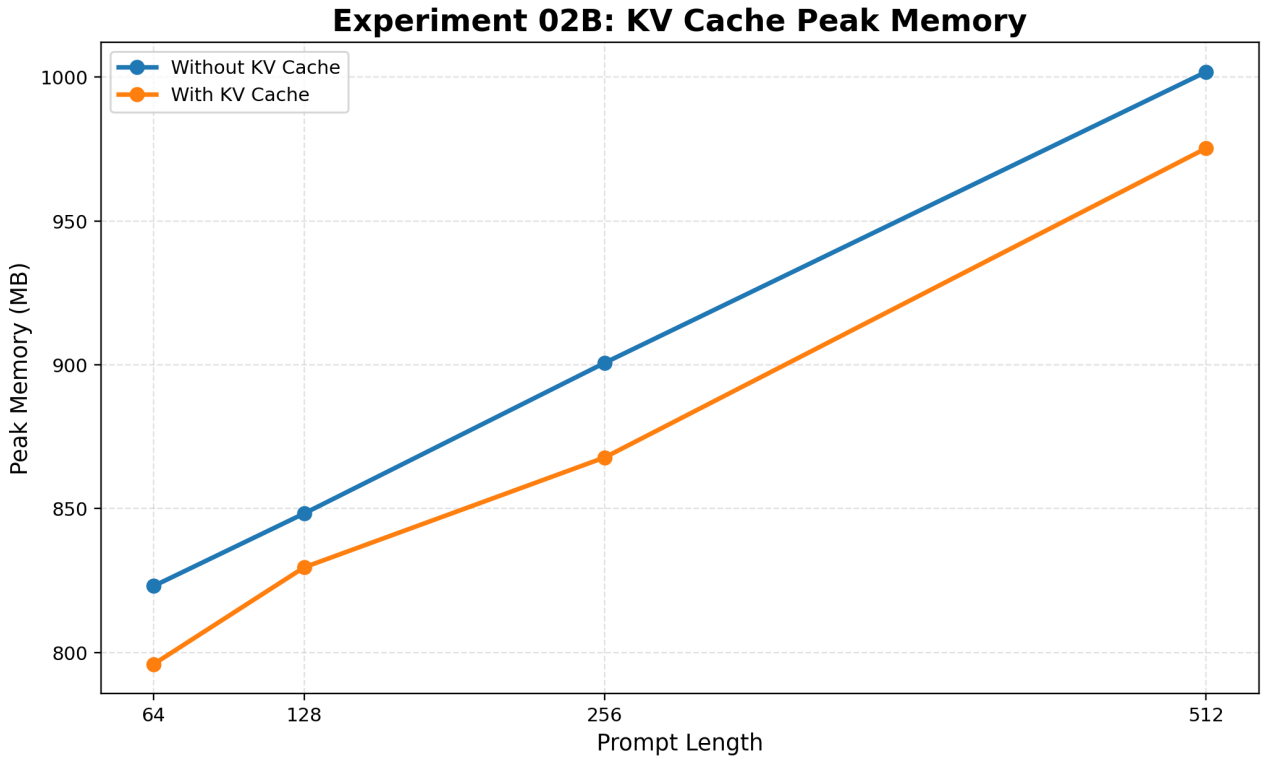

Figure 8. Peak memory comparison with and without KV-cache.

9. Result Analysis

Experiment 02B demonstrates that domain-specific tokenization can substantially improve language modeling quality on biomedical text. The final validation loss of 2.1301 and perplexity of 8.4161 are much stronger than the GPT-2/tiktoken medical baseline, supporting the hypothesis that tokenizer alignment with the domain improves modeling efficiency.

The KV-cache benchmark confirms that cached autoregressive decoding remains useful for this configuration. The benefit is most visible at longer prompt lengths, where reuse of previous attention states avoids repeated computation.

10. Future Work

A natural continuation is to evaluate whether the same custom-tokenizer setting can benefit from a more optimized attention implementation. This follow-up is treated as an efficiency-oriented extension rather than a new tokenizer study, allowing the project to separate modeling-quality improvements from systems-level optimization.

11. Conclusion

Experiment 02B successfully established a stronger medical-domain GPT training result using a custom biomedical BPE tokenizer. The model achieved stable convergence across five epochs, reached a final validation loss of 2.1301, and reduced perplexity to 8.4161. The experiment provides evidence that domain-specific tokenization is a major improvement over a generic GPT-2 tokenizer for biomedical language modeling.